MIP_PROMPT_LIBRARY_V2_PART1_DISCOVERY.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 1 - Discovery Prompts

Version: 2.0

Status: Approved

Purpose

Discovery is the foundation of MIP.

A modernization initiative should never begin with code conversion.

It should begin with understanding.

The objective of discovery is to identify:

- What exists
- How it interacts
- What depends on what
- What data flows through the system
- Where business capabilities reside

These prompts are designed to systematically extract that knowledge.

Prompt 01

Repository Discovery

File

prompts/discovery/repository_scan.md

Purpose

Perform a complete repository scan and create a factual inventory of all technical artifacts.

Context

Large mainframe repositories typically contain:

- COBOL Programs
- JCL
- PROCs
- Copybooks
- DB2 SQL
- VSAM Definitions
- CICS Definitions
- Utilities
- Documentation

Before any parser is built, engineers need visibility into repository contents.

Inputs

Repository root folder.

Instructions

Analyze the repository recursively.

Identify:

- COBOL Programs
- JCL Jobs
- Procedures
- Copybooks
- SQL Scripts
- Control Cards
- VSAM Definitions
- Documentation

Generate counts and inventories.

Highlight:

- Unknown file types
- Empty folders
- Missing documentation

Expected Output

Executive Summary

Repository Statistics

Artifact Type	Count

Repository Inventory

| Type | Name | Path | Description |

Discovery Risks

Recommendations

Constraints

Do not infer business functionality.

Use only observable evidence.

Success Criteria

Every repository artifact is cataloged and traceable.

Example Usage

Analyze the repository under mfcode and produce a complete artifact inventory.

Review Checklist

- All folders scanned?
- Counts accurate?
- Unknown files identified?
- Results reproducible?

Prompt 02

COBOL Program Inventory

File

prompts/discovery/program_inventory.md

Purpose

Build a complete inventory of COBOL programs.

Context

Programs are the primary execution units of most mainframe systems.

Understanding program inventory is the first step toward dependency analysis.

Inputs

COBOL source files.

Instructions

Extract:

- PROGRAM-ID
- Author
- Compilation Options
- Called Programs
- Copybooks
- Tables Referenced
- Files Referenced

Generate inventory and dependency summary.

Expected Output

Program Inventory

| Program | Path | Type | Description |

Dependency Summary

Missing Information

Constraints

Use source code evidence only.

Success Criteria

All COBOL programs are represented.

Example Usage

Analyze all COBOL members and generate an enterprise program inventory.

Review Checklist

- PROGRAM-ID extracted?
- Calls identified?
- Copybooks identified?
- Evidence retained?

Prompt 03

JCL Inventory

File

prompts/discovery/jcl_inventory.md

Purpose

Build a complete inventory of batch jobs and execution flows.

Context

JCL orchestrates enterprise batch processing.

Understanding jobs often reveals the true execution architecture.

Inputs

JCL members.

Instructions

Extract:

- JOB Names
- Steps
- EXEC Statements
- Procedures
- Datasets
- Utilities

Generate execution inventory.

Expected Output

Job Inventory

| Job | Step Count | Programs | Datasets |

Execution Summary

Dependency Summary

Constraints

Do not infer business meaning.

Success Criteria

Every batch job is represented.

Example Usage

Analyze all JCL members and build an execution inventory.

Review Checklist

- Steps identified?
- Programs identified?
- Procedures identified?
- Datasets identified?

Prompt 04

Copybook Inventory

File

prompts/discovery/copybook_inventory.md

Purpose

Build a complete inventory of shared data structures.

Context

Copybooks often contain the most valuable business knowledge in a legacy application.

Inputs

Copybook source files.

Instructions

Extract:

- Copybook Name
- Record Structures
- OCCURS Clauses
- REDEFINES Clauses
- Key Fields

Generate structural inventory.

Expected Output

Copybook Inventory

| Copybook | Records | Key Fields |

Structural Complexity Report

Reuse Analysis

Constraints

Do not attempt business interpretation.

Success Criteria

All copybooks represented consistently.

Example Usage

Analyze copybooks and identify shared enterprise data structures.

Review Checklist

- Structures extracted?
- OCCURS identified?
- REDEFINES identified?
- Complexity measured?

Prompt 05

DB2 Dependency Discovery

File

prompts/discovery/db2_dependency_discovery.md

Purpose

Identify all DB2 dependencies used throughout the application.

Context

Database dependencies are critical inputs into modernization planning.

Inputs

COBOL source files containing EXEC SQL.

Instructions

Extract:

- Tables
- Views
- Cursors
- Joins
- CRUD Operations
- Host Variables

Generate dependency inventory.

Expected Output

Table Inventory

| Table | Operation | Program |

DB2 Dependency Matrix

High-Risk Tables

Shared Tables

Constraints

Use SQL evidence only.

Success Criteria

All database dependencies are represented.

Example Usage

Analyze EXEC SQL statements and generate a DB2 dependency report.

Review Checklist

- Tables identified?
- Operations classified?
- Shared tables identified?
- Dependency matrix generated?

Part 1 Completion Criteria

Upon completion of these prompts, MIP should be capable of answering:

- What exists?
- Where does it exist?
- How many artifacts exist?
- Which programs execute?
- Which jobs orchestrate execution?
- Which copybooks define data?
- Which DB2 tables are used?

Only after these questions are answered should parsing and metadata extraction begin.

End of Part 1.